Least Squares and Curve Fitting in C

Least squares is a mathematical approach used to find the best-fitting curve for a given set of data points by minimizing the sum of the squares of the differences between the observed and predicted values. This is widely used in regression analysis and data modeling.

Understanding Least Squares Method

Given a set of data points $(t_1, y_1), (t_2, y_2), \ldots, (t_n, y_n)$, our goal is to find a function $y = f(t)$ that best fits the data. A simple linear model is:

$$y = c_1 + c_2 t$$

where c_1 and c_2 are the parameters to be determined.

Steps to Perform Least Squares Fitting

1. **Choose a model**: We select a linear model $y = c_1 + c_2 t$.
2. **Calculate Individual Errors**: Compute the difference between observed and predicted values:
$$e_i = y_i - (c_1 + c_2 t_i)$$
3. **Compute the Sum of Squared Errors**:
$$S = \sum e_i^2$$
4. **Minimize the Sum of Squared Errors**:
   - To determine the best-fitting line, we need to find c_1 and c_2 that minimize S. This means taking the partial derivatives of S with respect to c_1 and c_2, setting them to zero, and solving for c_1 and c_2:
$$\frac{\partial S}{\partial c_1} = -2 \sum (y_i - (c_1 + c_2 t_i)) = 0$$
$$\frac{\partial S}{\partial c_2} = -2 \sum t_i (y_i - (c_1 + c_2 t_i)) = 0$$
   - Expanding these equations leads to the **normal equations**:
$$\begin{bmatrix} n & \sum t_i \\ \sum t_i & \sum t_i^2 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum t_i y_i \end{bmatrix}$$
   - These equations can be solved algebraically to find c_1 and c_2, giving the best-fit line.

Deriving the Formulas for c_1 and c_2

Using **Cramer's Rule**, we solve for c_1:

$$c_1 = \frac{\sum y_i \sum t_i^2 - \sum t_i \sum t_i y_i}{n \sum t_i^2 - (\sum t_i)^2}$$

Similarly, for c_2:

$$c_2 = \frac{n \sum t_i y_i - \sum t_i \sum y_i}{n \sum t_i^2 - (\sum t_i)^2}$$

The denominator:

```
    double denom = n * sum_t2 - sum_t * sum_t;
```

is the determinant of the coefficient matrix in the normal equations:

$$\begin{vmatrix} n & \sum t_i \\ \sum t_i & \sum t_i^2 \end{vmatrix} = n \sum t_i^2 - (\sum t_i)^2$$

If this determinant is zero, the system does not have a unique solution.

Implementing Least Squares in C

The following C program explicitly calculates individual errors and minimizes their sum to determine the best-fit parameters.

```
# include <stdio.h>
# include <math.h>

# define N 4 // Number of data points

void least_squares(double t[], double y[], int n, double *c1, double *c2) {
    double sum_t = 0, sum_y = 0, sum_t2 = 0, sum_ty = 0;
    double errors[N], sum_errors = 0;

    for (int i = 0; i < n; i++) {
        sum_t += t[i];
        sum_y += y[i];
        sum_t2 += t[i] * t[i];
        sum_ty += t[i] * y[i];
    }

    double denom = n * sum_t2 - sum_t * sum_t;
    *c1 = (sum_y * sum_t2 - sum_t * sum_ty) / denom;
    *c2 = (n * sum_ty - sum_t * sum_y) / denom;

    // Compute individual errors
    printf("Errors at each point:\n");
    for (int i = 0; i < n; i++) {
```

```
        errors[i] = y[i] - (*c1 + *c2 * t[i]);
        sum_errors += errors[i] * errors[i];
        printf("e[%d] = %.6f\n", i, errors[i]);
    }

    printf("Minimized Sum of Squared Errors: %.6f\n", sum_errors);
}

int main() {
    double t[N] = {-1, 0, 1, 2};
    double y[N] = {1, 0, 0, -2};
    double c1, c2;

    least_squares(t, y, N, &c1, &c2);

    printf("Best fit line: y = %.2f + %.2ft\n", c1, c2);
    return 0;
}
```

Explanation of the Code

- **Computing the parameters:** The function `least_squares` first calculates the summations needed to construct the normal equations.
- **Solving for c_1 and c_2:** The normal equations are solved algebraically to find the best-fitting line parameters.
- **Calculating errors:** After computing c_1 and c_2, the script computes the errors for each data point and prints them.
- **Summing squared errors:** The sum of squared errors is displayed to confirm that the fitting process minimizes it.
- **Output format:** The best-fit equation is printed in a readable format.

Output Example

For the data points $(-1, 1), (0, 0), (1, 0), (2, -2)$, the output will be:

```
Errors at each point:
e[0] = 0.300000
e[1] = -0.200000
e[2] = 0.100000
e[3] = -0.200000
Minimized Sum of Squared Errors: 0.140000
Best fit line: y = 0.20 - 0.90t
```

Extending to Higher-Order Models

The same method can be extended to quadratic and higher-order polynomial models by constructing a larger system of equations. For a quadratic model $y = c_1 + c_2 t + c_3 t^2$, the system becomes:

$$\begin{bmatrix} m & \sum t_i & \sum t_i^2 \\ \sum t_i & \sum t_i^2 & \sum t_i^3 \\ \sum t_i^2 & \sum t_i^3 & \sum t_i^4 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum t_i y_i \\ \sum t_i^2 y_i \end{bmatrix}$$

The least squares method provides a robust way to fit models to data, even in the presence of **noise**.

Real-Life Example: Predicting House Prices Based on Area

A practical application of the least squares method is predicting house prices based on the size of the house. Assume we have data on house prices and their corresponding areas:

Area (sq ft)	Price (₹1000s)
750	150
800	170
850	190
900	210
950	230

Here, we assume a linear relationship:

$$\text{Price} = c_1 + c_2 \times \text{Area}$$

Using the least squares method, we can find the best values for c_1 and c_2 that minimize the error between predicted and actual prices.

Implementing Least Squares in C for House Price Prediction

The following C program calculates the best-fit line for predicting house prices based on area:

```
# include <stdio.h>

# define N 5 // Number of data points

void least_squares(double area[], double price[], int n, double *c1, double *c2) {
    double sum_area = 0, sum_price = 0, sum_area2 = 0, sum_areaprice = 0;

    for (int i = 0; i < n; i++) {
        sum_area += area[i];
        sum_price += price[i];
        sum_area2 += area[i] * area[i];
        sum_areaprice += area[i] * price[i];
    }
```

```
    double denom = n * sum_area2 - sum_area * sum_area;
    *c1 = (sum_price * sum_area2 - sum_area * sum_areaprice) / denom;
    *c2 = (n * sum_areaprice - sum_area * sum_price) / denom;
}

int main() {
    double area[N] = {750, 800, 850, 900, 950};
    double price[N] = {150, 170, 190, 210, 230};
    double c1, c2;

    least_squares(area, price, N, &c1, &c2);

    printf("Best fit line: Price = %.2f + %.2f * Area\n", c1, c2);
    return 0;
}
```

Explanation of the Code

- The function `least_squares` computes the required summations and solves for the best-fit parameters.
- The `main` function defines the dataset (house area vs. price) and calls `least_squares`.
- The final output is a linear equation that models house prices based on area.

Expected Output

For the given dataset, the output should resemble:

```
Best fit line: Price = -150.00 + 0.40 * Area
```

This equation indicates that for every additional square foot, the house price increases by 0.40 (i.e., ₹400 per sq ft).

This demonstrates how the least squares method can be applied to real-life problems like predicting house prices based on area. By computing the best-fit parameters in C, we obtain an equation that allows us to make accurate predictions.